

Module 1

Java Language, Key Attributes of Object-Oriented Programming, Java Development Kit, Simple Program, Create Blocks of Code, Keywords, Identifiers, The Java Class Libraries.

Data Types and Operators: Java's Primitive Types, Literals, Variables, Scope and Lifetime of Variables, Operators-Arithmetic, Relational, Logical, Bitwise, Assignment. Type conversion in Assignments, Using a Cast, Operator Precedence.

Program Control Structures: if, switch, for, enhanced for, while, do-while, break, continue.

Module 1

Object-oriented programming (OOP) is a programming paradigm that uses “Objects “and their interactions to design applications.

It simplifies the software development and maintenance by providing some concepts:

- ✓ **Object**
- ✓ **Class**
- ✓ **Data Abstraction & Encapsulation**
- ✓ **Inheritance**
- ✓ **Polymorphism**
- ✓ **Dynamic Binding**
- ✓ **Message Passing**

Module 1

In object-oriented programming technique, we design a program using objects and classes.

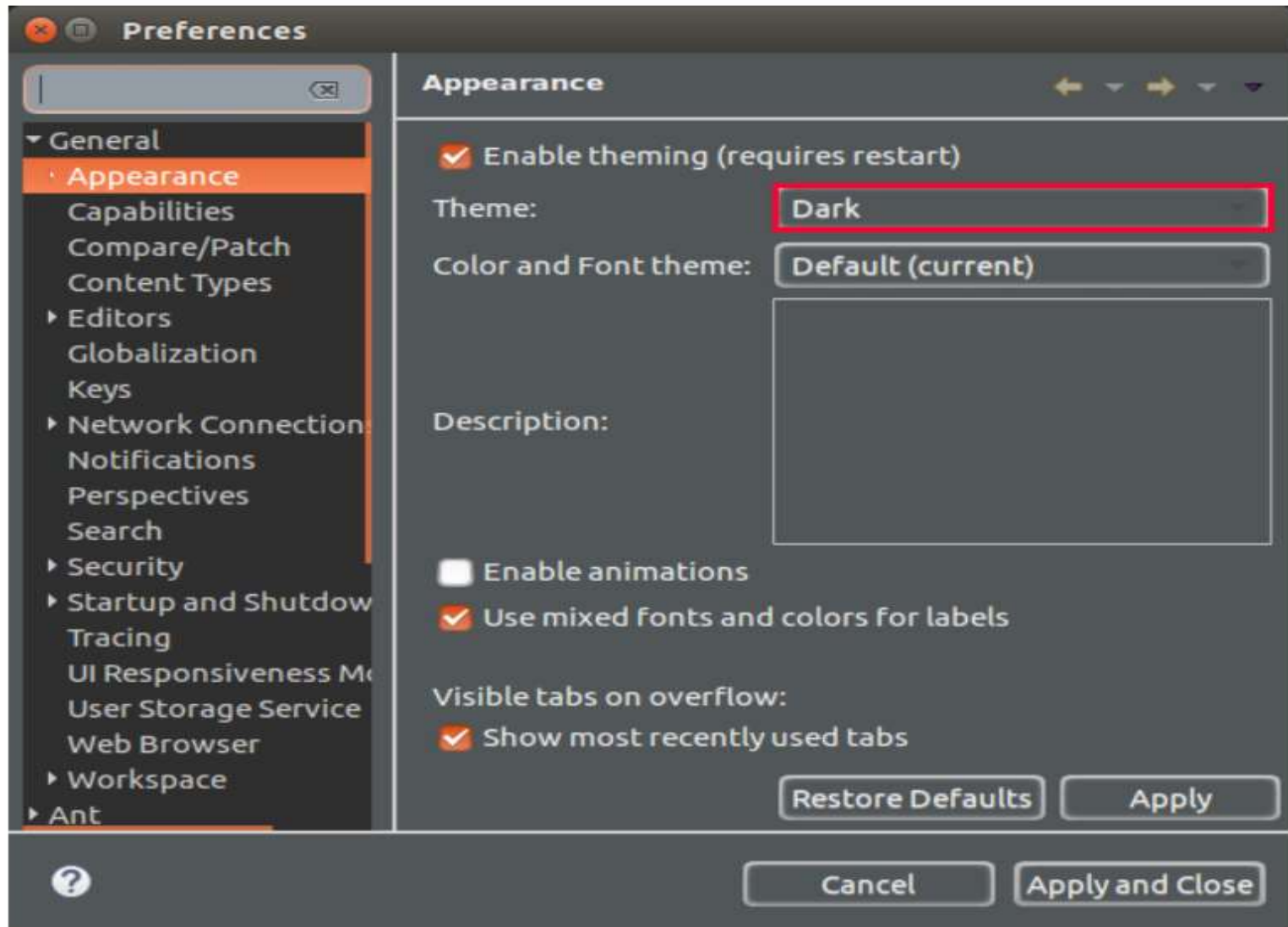
An object in Java is the physical as well as a logical entity, whereas, a class in Java is a logical entity only.

What is an object in Java

An entity that has state and behavior is known as an object e.g., chair, bike, marker, pen, table, car, etc. It can be physical or logical (tangible and intangible). The example of an intangible object is the banking system.

Eclipse

Eclipse.



Module 1

An object is an instance of a class. A class is a template or blueprint from which objects are created. So, an object is the instance(result) of a class.

Object Definitions:

An object is a real-world entity.

An object is a runtime entity.

The object is an entity which has state and behavior.

The object is an instance of a class.

CLASSES AND OBJECTS

A class is a group of objects which have common properties. It is a template or blueprint from which objects are created. It is a logical entity. It can't be physical.

A class in Java can contain:

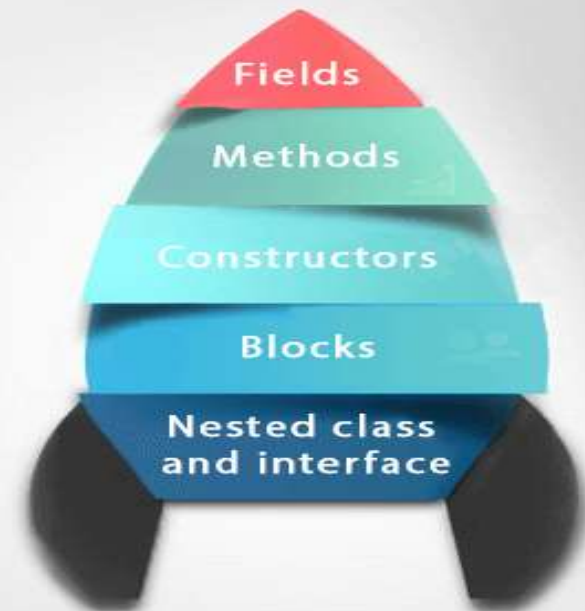
```
class <class_name>{
```

```
    field;
```

```
    method;
```

```
}
```

Class in Java



CLASSES AND OBJECTS

//Java Program to illustrate how to define a class and fields
//Defining a Student class.

```
class Student{  
    //defining fields  
  
    int id;           //field or data member or instance variable  
    String name;  
  
    //creating main method inside the Student class  
    public static void main(String args[]){  
        //Creating an object or instance  
        Student s1=new Student();//creating an object of Student  
        //Printing values of the object  
        System.out.println(s1.id);//accessing member through reference variable  
  
        System.out.println(s1.name);  
    }  
}
```

CLASSES AND OBJECTS

//Creating Student class.

```
class Student{  
int id;  
String name;  
}
```

//Creating another class TestStudent1 which contains the main method

```
class TestStudent1{  
public static void main(String args[]){  
    Student s1=new Student();  
    System.out.println(s1.id);  
    System.out.println(s1.name);  
}  
}
```

TestStudent1.java

javac

ABSTRACTION

- ✓ Abstraction is hiding the internal details and showing only essential functionality.
- ✓ In the abstraction concept, we do not show the actual implementation to the end user, instead we provide only essential things.
- ✓ Abstraction refers to the act of representing essential features without including the background details or explanations



ENCAPSULATION

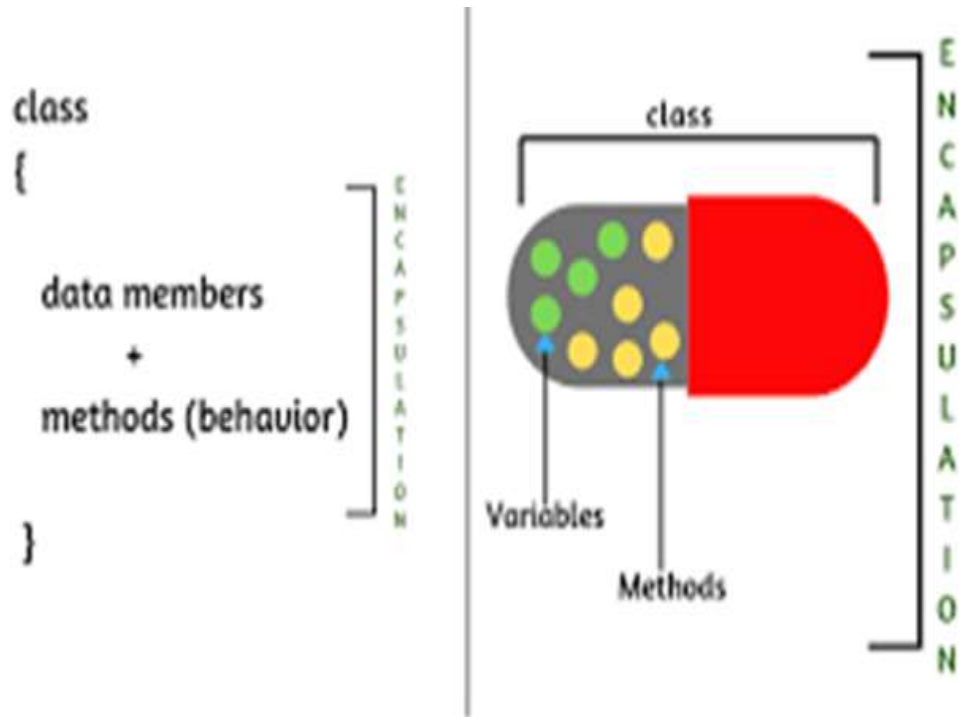


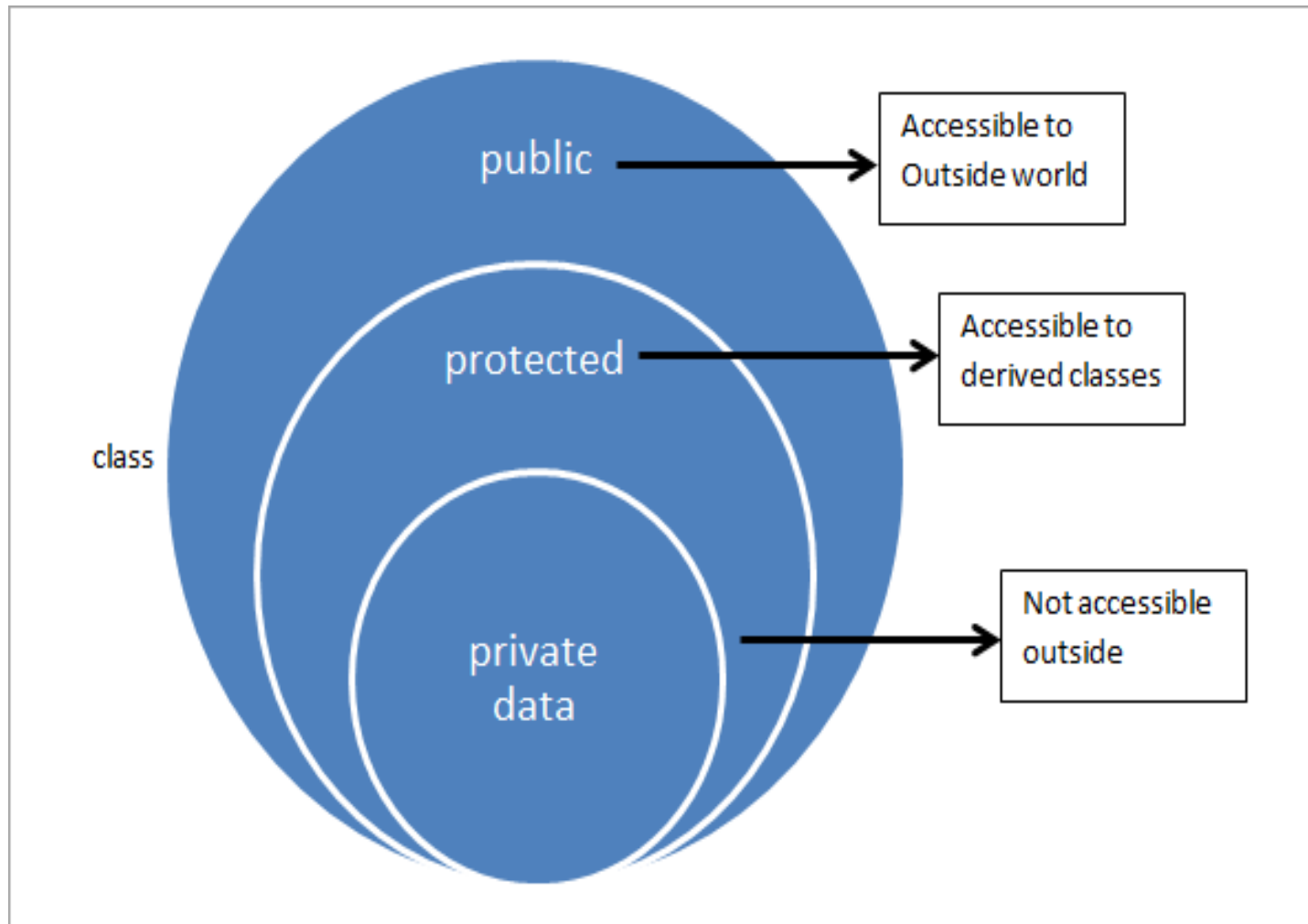
Fig: Encapsulation

- ✓ Encapsulation is the mechanism that binds together code and the data it manipulates

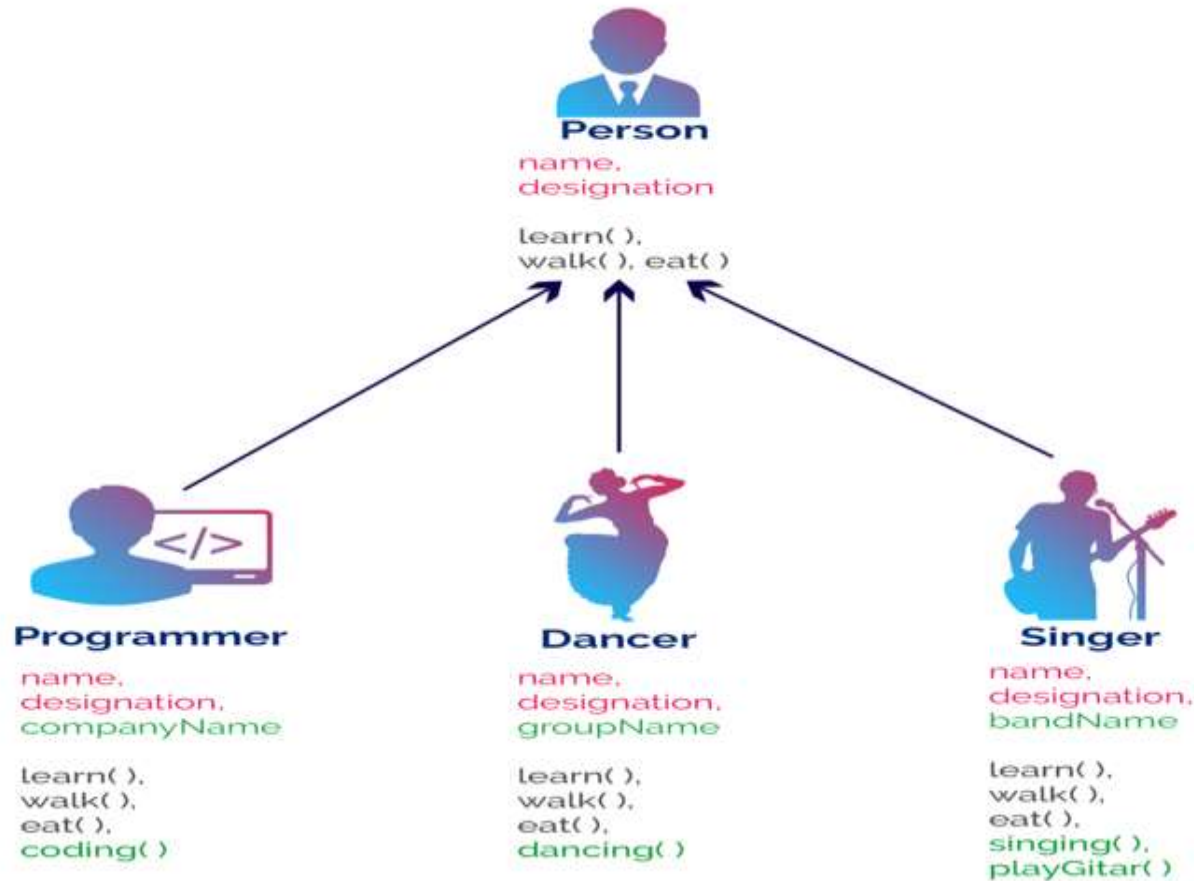
ENCAPSULATION

- ✓ Keeps both safe from outside interference and misuse.-DATA HIDING
- ✓ Access to the code and data inside the wrapper is tightly controlled through a well-defined interface
- ✓ Each method or variable in a class may be marked **private or public or protected**.
- ✓ The public interface of a class represents everything that external users of the class need to know, or may know.
- ✓ The private methods and data can only be accessed by code that is a member of the class

ENCAPSULATION



INHERITANCE



www.itechsmartclass.com

- ✓ Inheritance is the process of acquiring properties and behaviours from one class to another class.

INHERITANCE

- ✓ In inheritance, we derive a new class from the existing class.
- ✓ Here, the new class acquires the properties and behaviours from the existing class.
- ✓ In the inheritance concept, the class which provides properties is called as **parent class** and the class which receives the properties is called as **child class**.
- ✓ The parent class is also known as **base class** or **super class**. The child class is also known as **derived class** or **sub class**.
- ✓ In the inheritance, the properties and behaviors of base class extended to its derived class,

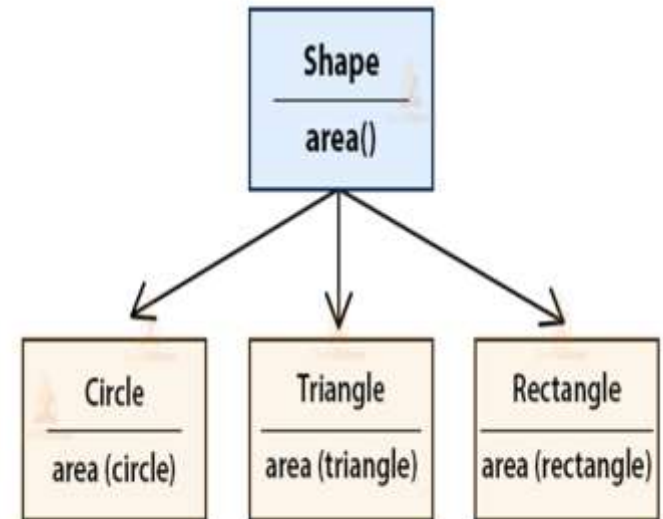
POLYMORPHISM

Polymorphism (from Greek, meaning “many forms”) is a feature that allows **one interface to be used for a general class of actions.**

The specific action is determined by the exact nature of the situation.

Polymorphism occur during compile time and runtime

Example of Polymorphism in Java



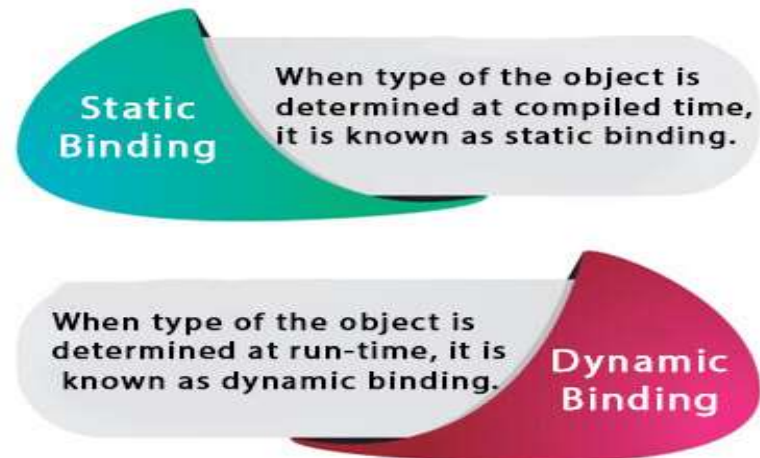
POLYMORPHISM

- ✓ Polymorphism in Java is the ability of an object to take many forms.
- ✓ Polymorphism in java allows us to perform the same action in many different ways

Two types of Polymorphism –

1. Overloading (ad hoc polymorphism, Static Binding)
2. Overriding (pure polymorphism, Dynamic Binding).

Static vs Dynamic Binding



Java Development Kit

- ✓ The Java Development Kit (JDK) is a software development environment which is used to develop java applications and applets
- ✓ JDK is an implementation of any one of the Java Platforms released by Oracle corporation:
 - **Standard Edition Java Platform** (Java SE) is a computing platform for development and deployment of portable code for desktop and server environments.)
 - **Enterprise Edition Java Platform**((Java EE) is the standard in community-driven enterprise software.)
 - **Micro Edition Java Platform** ((Java ME) is a computing platform for development and deployment of portable code for embedded and mobile devices (micro-controllers, sensors, gateways, mobile phones, personal digital assistants, TV set-top boxes, printers))

Components of JDK

appletviewer:	This tool is used to run and debug Java applets without a web browser.
apt:	It is an annotation-processing tool.
extcheck:	It is a utility that detects JAR file conflicts.
idlj:	An IDL-to-Java compiler. This utility generates Java bindings from a given Java IDL file.
jabswitch:	It is a Java Access Bridge. Exposes assistive technologies on Microsoft Windows systems.
java:	The loader for Java applications. This tool is an interpreter and can interpret the class files generated by the javac compiler. Now a single launcher is used for both development and deployment. The old deployment launcher, jre, no longer comes with Sun JDK, and instead it has been replaced by this new java loader.
javac:	It specifies the Java compiler, which converts source code into Java bytecode.
javadoc:	The documentation generator, which automatically generates documentation from source code comments
jar:	The specifies the archiver, which packages related class libraries into a single JAR file. This tool also helps manage JAR files.
javafxpackager:	It is a tool to package and sign JavaFX applications.
jarsigner:	the jar signing and verification tool.
javah:	the C header and stub generator, used to write native methods.
javap:	the class file disassembler.
javaws:	the Java Web Start launcher for JNLP applications.
JConsole:	Java Monitoring and Management Console.
jdb:	the debugger.
jhat:	Java Heap Analysis Tool (experimental).
jinfo:	This utility gets configuration information from a running Java process or crash dump.
jmap:	Oracle jmap - Memory Map- This utility outputs the memory map for Java and can print shared object memory maps or heap memory details of a given process or core dump.

Components of JDK

jmc:	Java Mission Control
jps:	Java Virtual Machine Process Status Tool lists the instrumented HotSpot Java Virtual Machines (JVMs) on the target system.
jrunscript:	Java command-line script shell.
jstack:	It is a utility that prints Java stack traces of Java threads (experimental).
jstat:	Java Virtual Machine statistics monitoring tool (experimental).
jstatd:	jstat daemon (experimental).
keytool:	It is a tool for manipulating the keystore.
pack200:	JAR compression tool.
Policytool:	It specifies the policy creation and management tool, which can determine policy for a Java runtime, specifying which permissions are available for code from various sources.
VisualVM:	It is a visual tool integrating several command-line JDK tools and lightweight [clarification needed] performance and memory profiling capabilities
wsimport:	It generates portable JAX-WS artifacts for invoking a web service.
xjc:	It is the part of the Java API for XML Binding (JAXB) API. It accepts an XML schema and generates Java classes.

Create Blocks of Code

A code block is a section of code enclosed in curly braces `{}`. Code blocks are used to group one or more statements. Code blocks can be nested, meaning you can have code blocks inside of other code blocks.

```
{  
    int x = 10;  
    int y = 20;  
    System.out.println(x + y);  
}
```

```
{  
    int x = 10;  
}
```

Create Blocks of Code

Nested code blocks

You can also nest code blocks inside of other code blocks. Nested code blocks are often used to group related statements.

```
public class Main {  
    public static void main(String[] args) {  
        // this is the main method  
        int x = 10;  
        int y = 20;  
  
        // this code block is nested inside the main method  
        {  
            // print out the sum of x and y  
            System.out.println(x + y);  
  
            // print out the difference of x and y  
            System.out.println(x - y);  
        }  
    }  
}
```

List of Java Keywords

Primitive Types and void	Modifiers	Declarations	Control Flow	Miscellaneous
1.boolean	1.public	1.class	1.if	1.this
2.byte	2.protected	2.interface	2.else	2.new
3.char	3.private	3.enum	3.try	3.super
4.short	4.abstract	4.extends	4.catch	4.import
5.int	5.static	5.implements	5.finally	5.instanceof
6.long	6.final	6.package	6.do	6.null
7.float	7.transient	7throws	7.while	7.true
8.double	8.volatile		8.for	8.false
9.void	9.synchronized		9.continue	9.strictfp
	10.native		10.break	10.assert
			11.switch	11._ (underscore)
			12.case	12.goto
			13.default	13.const
			14.throw	
			15.return	

Identifiers

In programming languages, identifiers are used for identification purposes. In Java, an identifier can be a class name, method name, variable name, or label

```
public class Test
{
    public static void main(String[] args)
    {
        int a = 20;
    }
}
```

we have 5 identifiers namely :

Test : class name.

main : method name.

String : predefined class name.

args : variable name.

a : variable name.

Rules for defining Java Identifiers

- ✓ The only allowed characters for identifiers are all alphanumeric characters([A-Z],[a-z],[0-9]), '\$'(dollar sign) and '_' (underscore).For example "gitam@" is not a valid java identifier as it contain '@' special character.
- ✓ Identifiers should not start with digits([0-9]). For example "123geeks" is a not a valid java identifier.
- ✓ Java identifiers are case-sensitive.
- ✓ There is no limit on the length of the identifier but it is advisable to use an optimum length of 4 – 15 letters only.
- ✓ Reserved Words can't be used as an identifier. For example "int while = 20;" is an invalid statement as while is a reserved word. There are 53 reserved words in Java.

Examples of invalid identifiers :

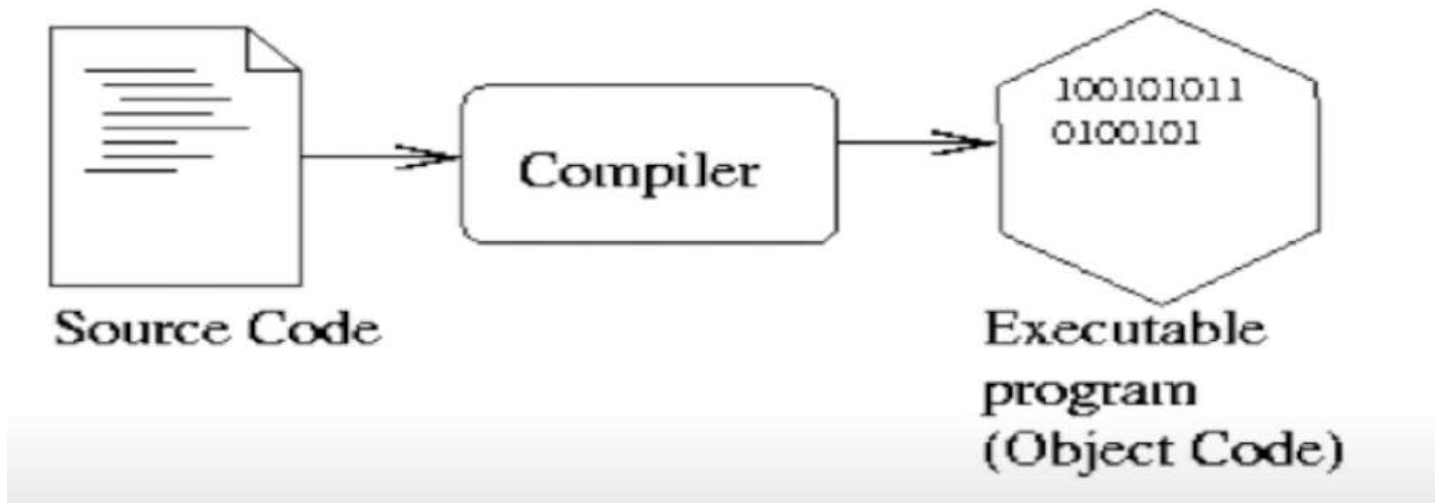
Rules for defining Java Identifiers

Examples of invalid identifiers :

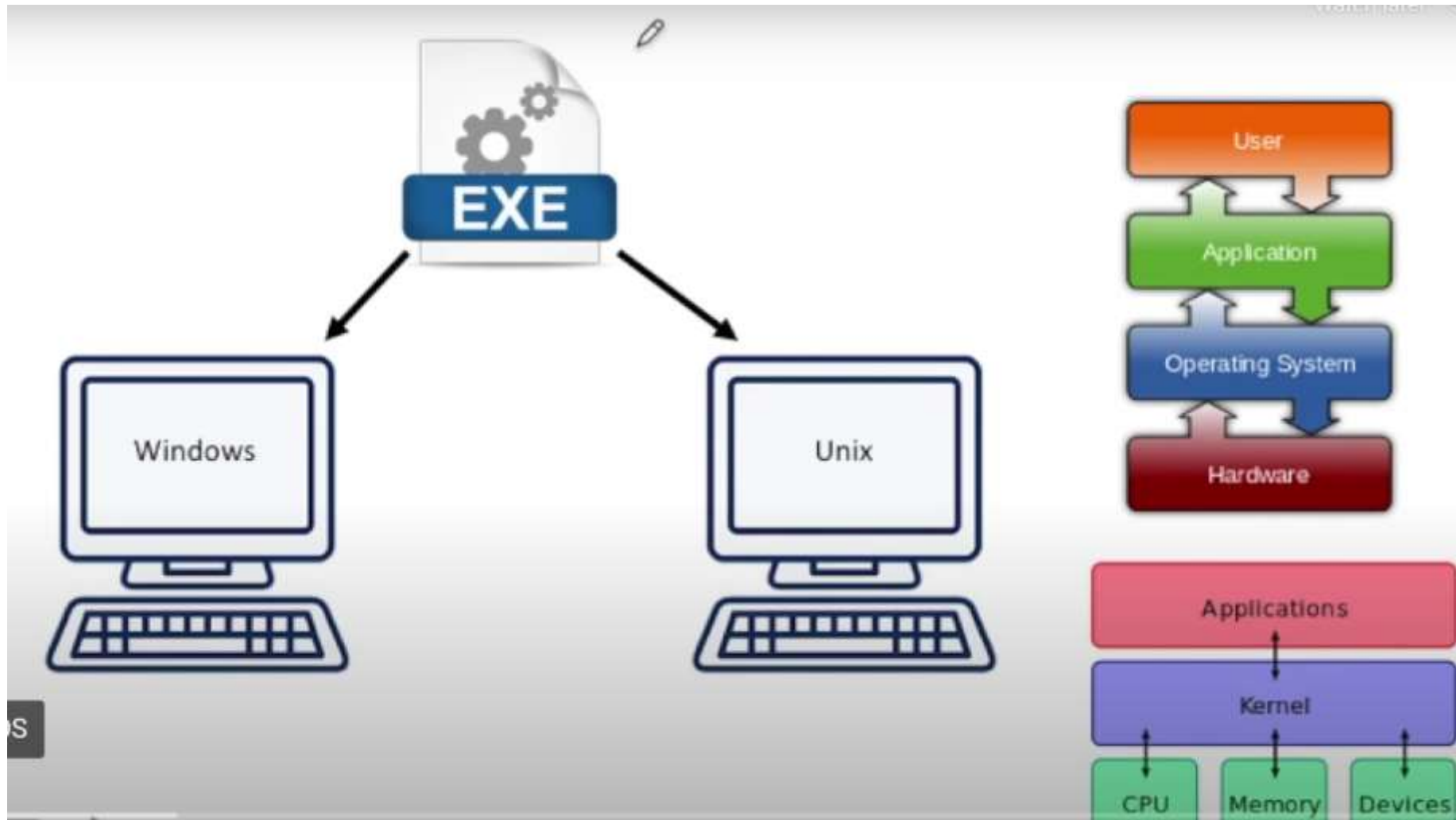
- ✓ My Variable // contains a space
- ✓ 123geeks // Begins with a digit
- ✓ a+c // plus sign is not an alphanumeric character
- ✓ variable-2 // hyphen is not an alphanumeric character
- ✓ sum_&_difference // ampersand is not an alphanumeric character

```
// Java program to demonstrate working
// of java.lang.Math.sqrt() method
import java.lang.Math;
class Main {
    public static void main(String args[])
    { double a = 30;
      System.out.println(Math.sqrt(a));
      a = 45;
      System.out.println(Math.sqrt(a));
      a = 60;
      System.out.println(Math.sqrt(a));
    }
}
```

Compilation and Execution of a Java Program



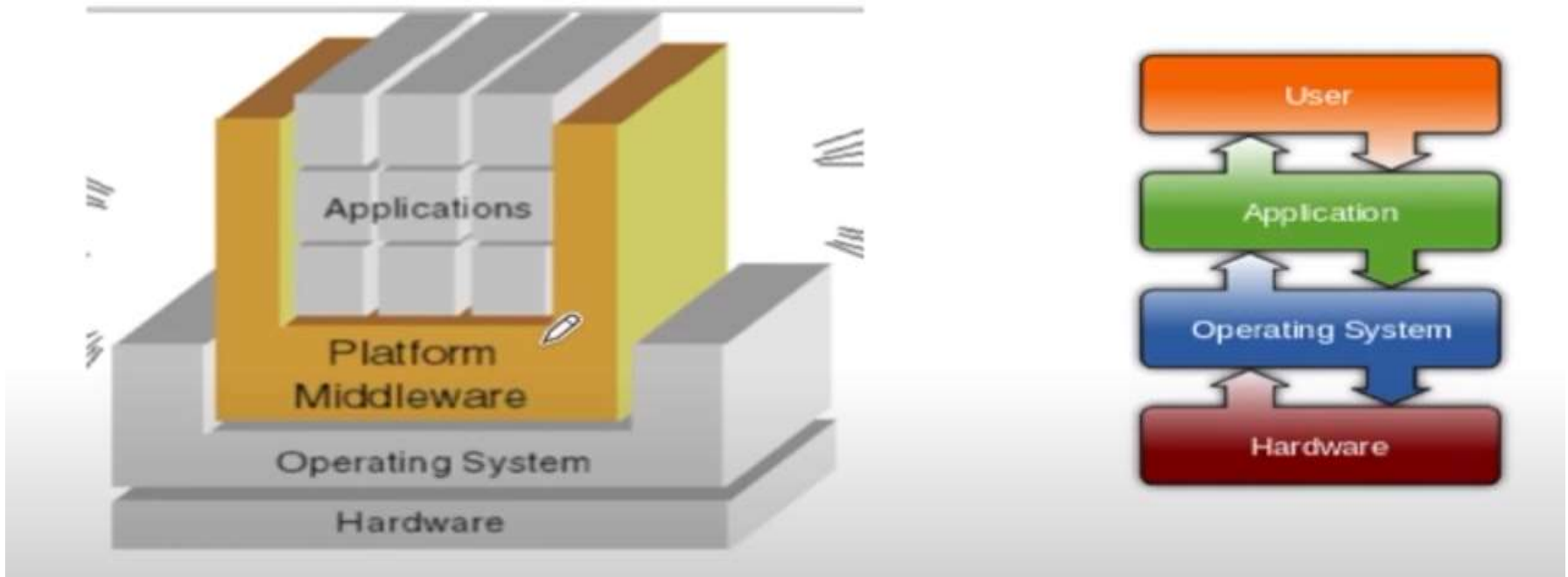
Compilation and Execution of a Java Program



Compilation and Execution of a Java Program

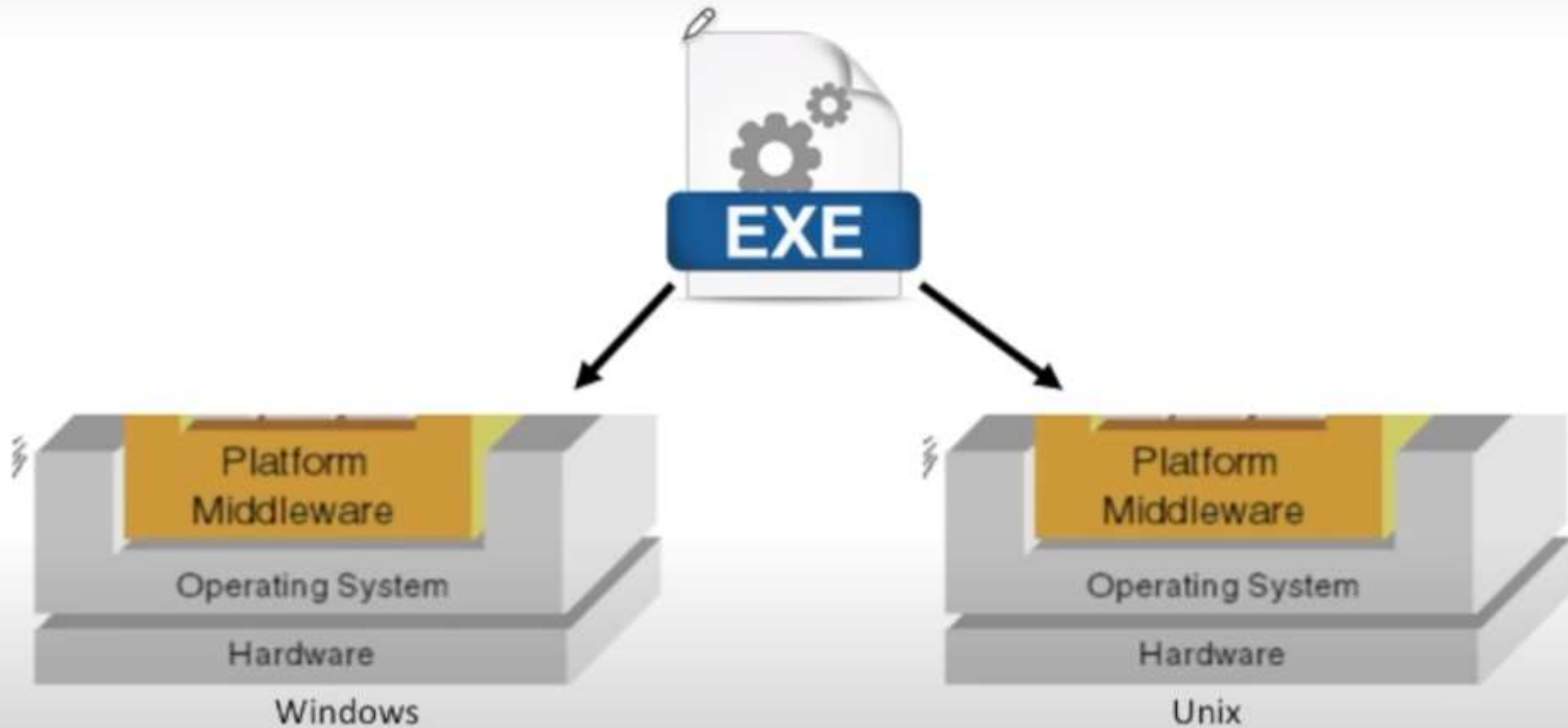


Compilation and Execution of a Java Program



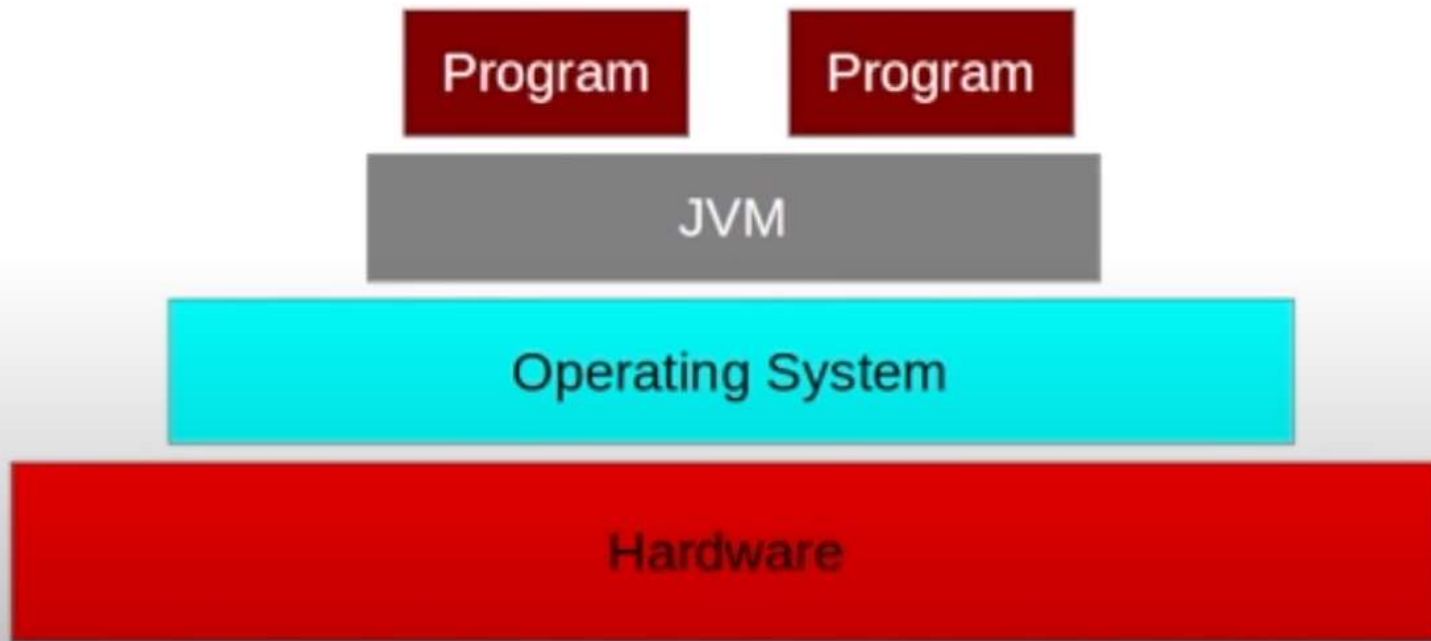
- Java, being a platform-independent programming language, doesn't work on the one-step compilation.
- it involves a two-step execution, *first through an OS-independent compiler*; and *second, in a virtual machine (JVM)* which is custom-built for every operating system.

Compilation and Execution of a Java Program

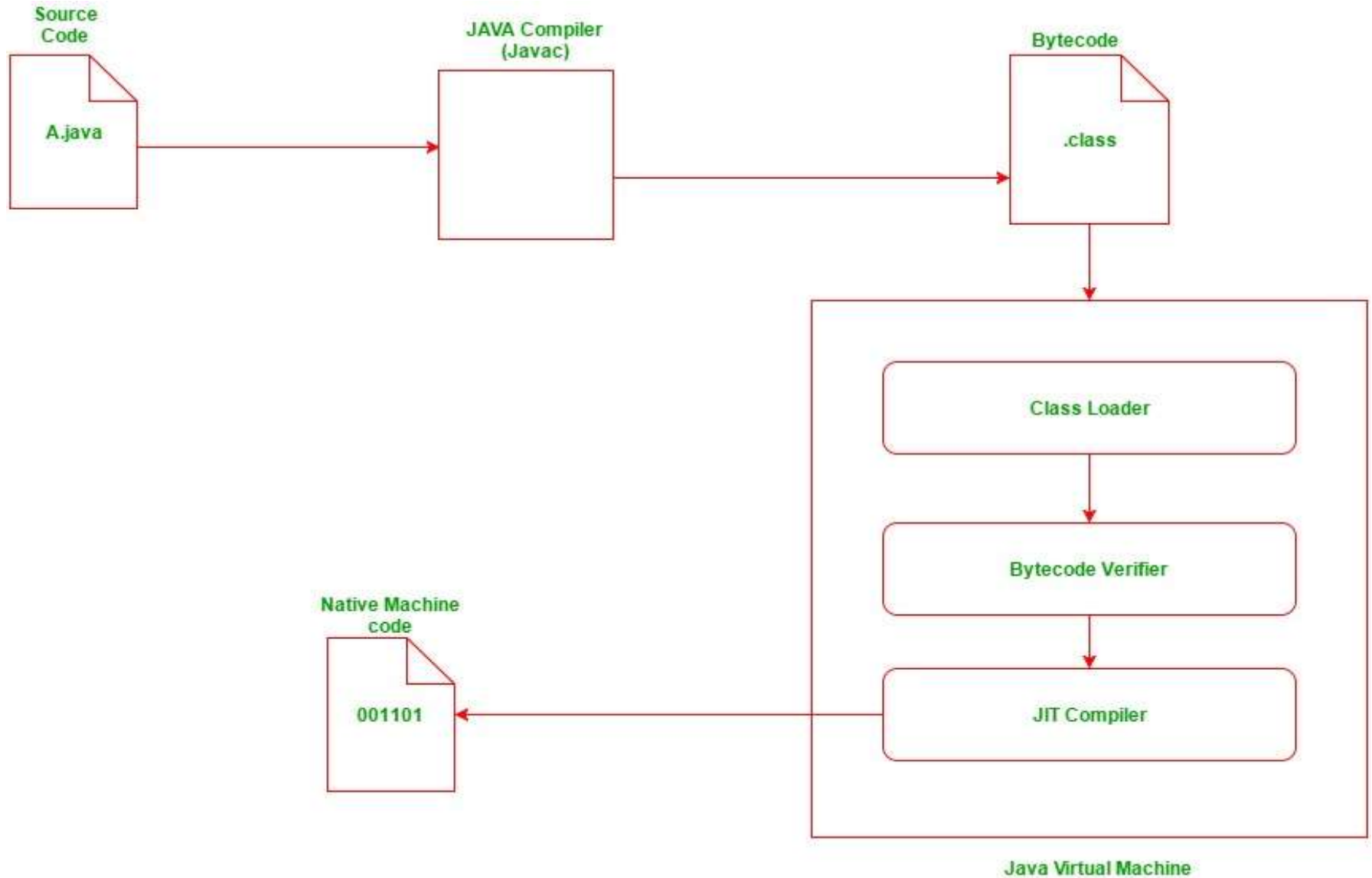


Compilation and Execution of a Java Program

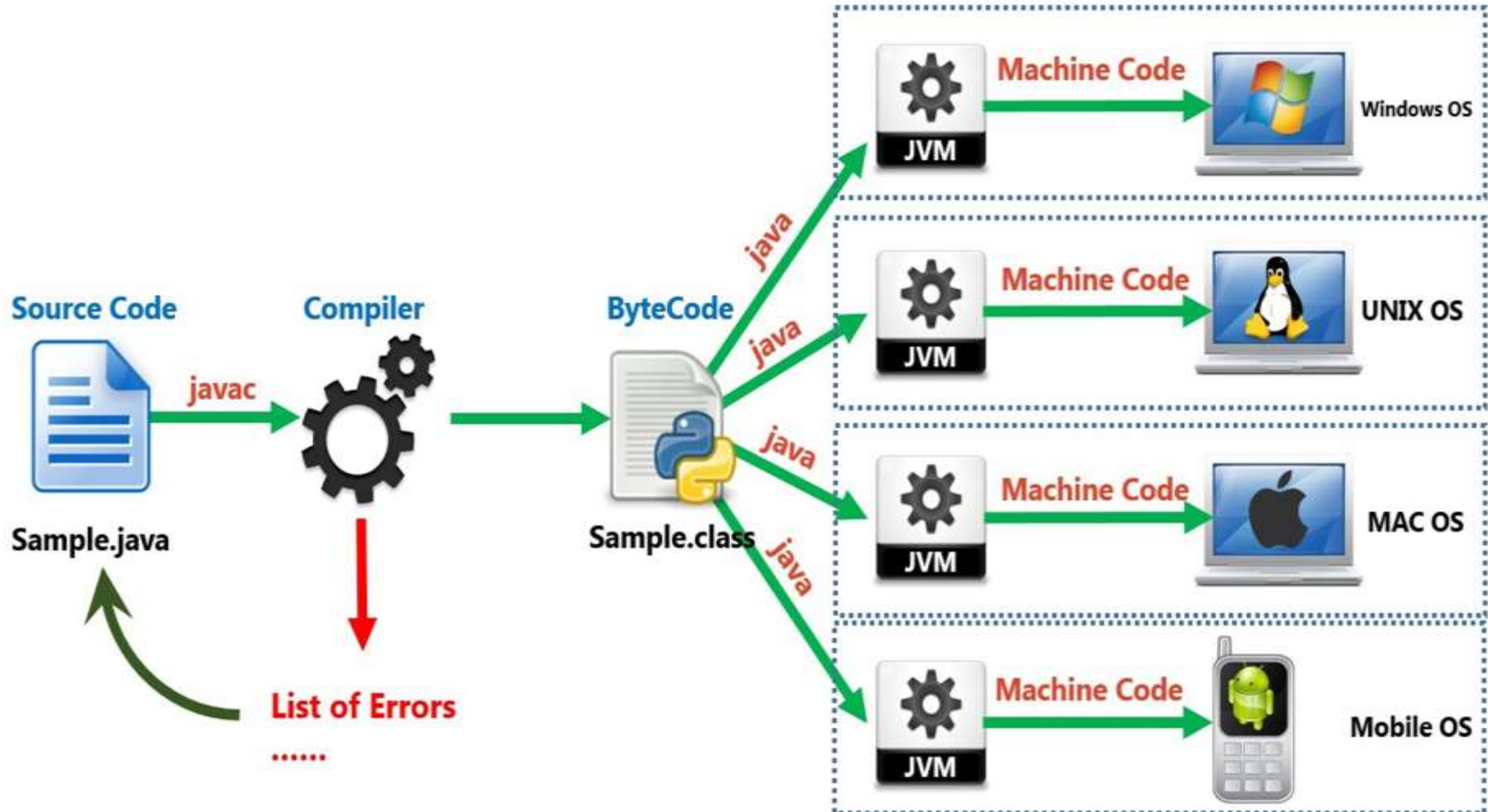
Java Virtual Machine (VM)



Compilation and Execution of a Java Program



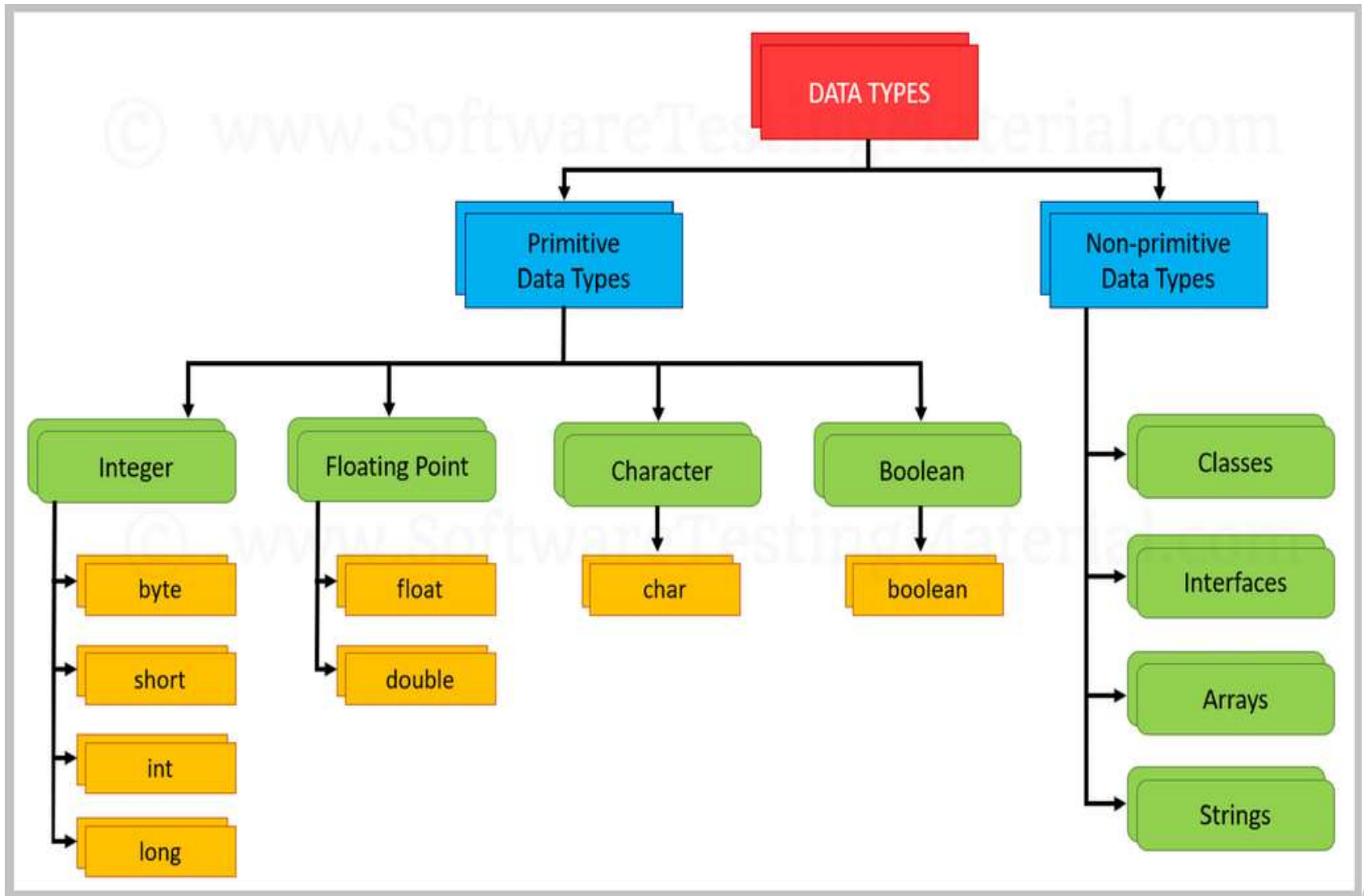
Compilation and Execution of a Java Program



Data Types

```
public class Main {  
    public static void main(String[] args) {  
        int myNum = 5;  
        float myFloatNum = 5.99f;  
        char myLetter = 'D';  
        boolean myBool = true;  
        String myText = "Hello";  
        System.out.println(myNum);  
        System.out.println(myFloatNum);  
        System.out.println(myLetter);  
        System.out.println(myBool);  
        System.out.println(myText);  
    }  
}
```

Data Types



Data Types

Primitive Data Types

A primitive data type specifies the size and type of variable values, and it has no additional methods.

There are eight primitive data types in Java:

Data Type	Size	Description
<code>byte</code>	1 byte	Stores whole numbers from -128 to 127
<code>short</code>	2 bytes	Stores whole numbers from -32,768 to 32,767
<code>int</code>	4 bytes	Stores whole numbers from -2,147,483,648 to 2,147,483,647
<code>long</code>	8 bytes	Stores whole numbers from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
<code>float</code>	4 bytes	Stores fractional numbers. Sufficient for storing 6 to 7 decimal digits
<code>double</code>	8 bytes	Stores fractional numbers. Sufficient for storing 15 decimal digits
<code>boolean</code>	1 bit	Stores true or false values
<code>char</code>	2 bytes	Stores a single character/letter or ASCII values

Variable Scope

scope of variable defines how a specific variable is accessible within the program or across classes a variable can be declared and defined inside a class, method, or block. It defines the scope of the variable i.e. the visibility or accessibility of a variable. Variable declared inside a block or method are not visible to outside. If we try to do so, we will get a compilation error.

There are three types of variables.

- ✓ Instance Variables
- ✓ Class/static Variables
- ✓ Local Variables

Scope of a variable refers to in which areas or sections of a program can the variable be accessed

lifetime of a variable refers to how long the variable stays alive in memory.

Variable Scope

Java Local Variables

- Local variables are declared in methods, constructors, or blocks.
- Local variables are created when the method, constructor or block is entered and the variable will be destroyed once it exits the method, constructor, or block.
- Access modifiers cannot be used for local variables.
- Local variables are visible only within the declared method, constructor, or block.
- Local variables are implemented at stack level internally.
- There is no default value for local variables, so local variables should be declared and an initial value should be assigned before the first use.

Variable Scope

```
public class Test {  
    public void pupAge() {  
        int age = 0;  
        age = age + 7;  
        System.out.println("Puppy age is : " + age);  
    }  
  
    public static void main(String args[]) {  
        Test test = new Test();  
        test.pupAge();  
    }  
}
```


Variable Scope

Variable's local scope without initialization—

```
public class Test {  
    public void pupAge() {  
        int age;  
        age = age + 7;  
        System.out.println("Puppy age is : " + age);  
    }  
    public static void main(String args[]) {  
        Test test = new Test();  
        test.pupAge();  
    }  
}
```

```
1 Test.java:4:variable number might not have been initialized  
2 age = age + 7;  
3           ^  
4 1 error
```

Variable Scope:**Instance Variables:**

Instance Variables:

- The variables that are declared inside the class but outside the scope of any method are called instance variables in Java.
- The instance variable is initialized at the time of the class loading or when an object of the class is created.
- An instance variable can be declared using different access modifiers available in Java like default, private, public, and protected.
- Instance variables of different types have default values that are specified in the next point.

Variable Scope:**Instance Variables:**

Features

- 1.To use an instance variable an object of the class must be created.
- 2.An instance variable is destroyed when the object it is associated with is destroyed.
- 3.An instance variable does not compulsory need to be initialized.
- 4.Instance variables are accessible inside the same class that declares them.

Variable Scope:**Instance Variables:**

Limitations of Instance Variable

- 1.It cannot be declared static, abstract, synchronized, and native.
- 2.It can be declared final and transient.
- 3.It can be of any of the four access modifiers available in Java (private, public, protected, and default).

Variable Scope:Instance Variables:

```
public String name;  
    private double salary; // salary variable is visible in Employee class only.  
    public Employee (String empName) {  
        name = empName; // The name variable is assigned in the constructor.  
    }  
  
    public void setSalary(double empSal) {  
        salary = empSal; // The salary variable is assigned a value.  
    }  
  
    public void printEmp() {  
        System.out.println("name : " + name ); // This method prints the employee details.  
        System.out.println("salary :" + salary);  
    }  
  
    public static void main(String args[]) {  
        Employee empOne = new Employee("Ransika");  
        empOne.setSalary(1000);  
        empOne.printEmp();  
    }  
}
```

Variable Scope:Instance Variables:

Default Values of Instance Variables in Java

Instance variable type	Default values
boolean	false
byte	(byte) 0
short	(short) 0
int	0
double	0.0d
float	0.0
long	0L
Object	null
char	\u0000

Variable Scope: **Instance Variables:**

```
public class Studentsrecords
{
    /* declaration of instance variables. */
    public String name;           //public instance
    String division;           //default instance
    private int age;           //private instance
    /* Constructor that initialize an instance variable. */
    public Studentsrecords(String sname)
    {
        name = sname;    }
    /* Method to initialize an instance variable. */
    public void setDiv(String sdiv)
    {
        division = sdiv;    }
    /* Method to initialize an instance variable. */
    public void setAge(int sage)
    {
        age = sage;    }
```

Variable Scope:Instance Variables:

```
/* Method to display the values of instance variables. */  
public void printstud()  
{  
    System.out.println("Student Name: " + name );  
    System.out.println("Student Division: " + division);  
    System.out.println("Student Age: " + age);  
}
```

/* Driver Code */

```
public static void main(String args[])  
{  
    Studentsrecords s = new Studentsrecords("Monica");  
    s.setAge(14);  
    s.setDiv("B");  
    s.printstud();  
}
```

Output:

```
Student Name: Monica  
Student Division: B  
Student Age: 14
```


✓ Variable Scope: **Class Variables**

- Class variables are also known as ***static variables***, and they are declared outside a method, with the help of the keyword '***static***'.
- Static variable is the one that is common to all the instances of the class. A single copy of the variable is shared among all objects.

Variable Scope

Local Variables

All variables which are not instance or class variables are known as local variables.

Scope: Within the block it is declared.

Lifetime: Until control leaves the block in which it is declared.

Variable Type	Scope	Lifetime
Instance variable	Troughout the class except in static methods	Until the object is available in the memory
Class variable	Troughout the class	Until the end of the program
Local variable	Within the block in which it is declared	Until the control leaves the block in which it is declared

Variable Scope

```
public class scope_and_lifetime {  
    int num1, num2; //Instance Variables  
    static int result; //Class Variable  
    int add(int a, int b){ //Local Variables  
        num1 = a;  
        num2 = b;  
        return a+b;  
    }  
    public static void main(String args[]){  
        scope_and_lifetime ob = new scope_and_lifetime();  
        result = ob.add(10, 20);  
        System.out.println("Sum = " + result);  
    }  
}
```

Variable Scope

```
public class scope_and_lifetime {  
    int num1, num2; //Instance Variables  
    int result;  
    int add(int a, int b){ //Local Variables  
        num1 = a;  
        num2 = b;  
        return a+b;  
    }  
    public static void main(String args[]){  
        scope_and_lifetime ob = new scope_and_lifetime();  
        result = ob.add(10, 20);  
        num1 = 10;  
        System.out.println("Sum = " + result);  
    }  
}
```

removed the keyword *static* from the variable *result*, which makes it an *instance variable*. We cannot use *instance variable* inside a *static* method, so the usage of *num1* inside *public static void main(String args[])* gives an error.

Variable Scope

Nested Scope: We can always use a block of code inside a block of code

```
public class scope_and_lifetime {  
    public static void main(String args[]){  
        int a = 10;  
        //Outer Block  
        {  
            //Inner Block  
            int x = a;  
            {  
                int y = x;  
            }  
        }  
        System.out.println(y);  
    }  
}
```

Variable Scope

```
public class VariableScopeExample1
{
    public static void main(String args[])
    {
        int x=10;
        {
            //y has limited scope to this block only
            int y=20;
            System.out.println("Sum of x+y = " + (x+y));
        }
        //here y is unknown
        y=100;
        //x is still known
        x=50;
    }
}
```

Local Variables (Method Level Scope)

These are the variables that are declared inside a method, constructor, or block have a method-level or block-level scope and cannot be accessed outside in which it is defined. Variables declared inside a pair of curly braces {} have block-level scope.

```
public class DemoClass1
{
    void show()
    { //variable declared inside a method has method level scope
        int x=10;
        System.out.println("The value of x is: "+x);
    }
    public static void main(String args[])
    {
        DemoClass1 dc = new DemoClass1();
        dc.show();
    }
}
```

Local Variables (Method Level Scope)

```
public class DemoClass2
{
    private int a;
    public void setNumber(int a)
    {
        this.a = a;
        System.out.println("The value of a is: "+a);
    }
    public static void main(String args[])
    {
        DemoClass2 dc = new DemoClass2();
        dc.setNumber(3);
    }
}
```

this keyword that differentiates the class variable and local variable

Local Variables (Method Level Scope)

```
public class VariableInsideConstructor
{
VariableInsideConstructor() //creating a default constructor
{ int age=24;
  System.out.println("Age is: "+age);
}

public static void main(String args[]) //main() method
{
//calling a default constructor
VariableInsideConstructor vc=new VariableInsideConstructor();

}
}
```

Local Variables (Method Level Scope)

```
public class VariableInsideBlock
{
    public static void main(String args[])
    {
        int x=4;
        {
            int y=100; //y has limited scope to this block only
            System.out.println("Sum of x+y = " + (x+y));
            y=10;
            int y=200; //gives error, already defined
        }
        //creates a new variable
    }
}
```

```
int y;
```

Output:

```
/VariableScopeExample2.java:12: error: variable y is already defined in method main(String[])
    int y=200;
      ^
1 error
```

```
public class BlockScopeExample1
{
    public static void main(String args[])
    {
        for (int x = 0; x < 10; x++)
        { System.out.println(x);
        }
        System.out.println(x);
    }
}
```

Output:

```
/BlockScopeExample.java:9: error: cannot find symbol
System.out.println(x);
                   ^
symbol:   variable x
location: class BlockScopeExample
1 error
```

```
public class BlockScopeExample2
{
    public static void main(String args[])
    {
        int x;
        for (x = 0; x < 10; x++)
        {
            System.out.print(x+"\t"); //prints 0 to 9
        }

        System.out.println(x); //prints 10
    }
}
```

Java Operators

Operators are used to perform operations on variables and values.

- ✓ Unary Operator
- ✓ Arithmetic operators
- ✓ Assignment operators
- ✓ Comparison operators/Relational Operator,
- ✓ Logical operators
- ✓ Shift Operator
- ✓ Ternary Operator and
- ✓ Bitwise operators

Unary Operator

Types of Unary Operators in Java

There are three types of unary operators in java. They are as follows:

- ✓ Unary minus operator (`-`)
- ✓ Increment operator (`++`)
- ✓ Decrement operator (`--`)

```
public class MinusExample
{ public static void main(String [] args)
  { int x = 5;
    x = -5;
    System.out.println(x);
  }
}
```

Java Operators

```
public class OpTest1
{ public static void main(String[] args)
  { int x = 50; int y = 100; int z = 200;  int a, b, c;
    a = ++x;    b = y++;
    c = x + y++ + ++z;
    System.out.println("x = " +x);
    System.out.println("y = " +y);
    System.out.println("z = " +z);
    System.out.println("a = " +a);
    System.out.println("b = " +b);
    System.out.println("c = " +c);
  }
}
```

Arithmetic operators

Arithmetic Operators :Arithmetic operators are used to perform common mathematical operations

Operator	Name	Description	Example
+	Addition	Adds together two values	$x + y$
-	Subtraction	Subtracts one value from another	$x - y$
*	Multiplication	Multiplies two values	$x * y$
/	Division	Divides one value by another	x / y
%	Modulus	Returns the division remainder	$x \% y$
++	Increment	Increases the value of a variable by 1	<code>++x</code>
--	Decrement	Decreases the value of a variable by 1	<code>--x</code>

Arithmetic operators

```
public class Main {  
    public static void main(String[] args) {  
        int x = 12;  
        int y = 3;  
        System.out.println(x / y);  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        int x = 5;  
        int y = 2;  
        System.out.println(x % y);  
    }  
}
```

Arithmetic operators

```
class Main {  
    public static void main(String[] args) {  
        int a = 12, b = 5; // declare variables  
        System.out.println("a + b = " + (a + b)); // addition operator  
        System.out.println("a - b = " + (a - b)); // subtraction operator  
        System.out.println("a * b = " + (a * b)); // multiplication operator  
        System.out.println("a / b = " + (a / b)); // division operator  
        System.out.println("a % b = " + (a % b)); // modulo operator  
    }  
}
```

Assignment operators

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
&=	x &= 3	x = x & 3
=	x = 3	x = x 3
^=	x ^= 3	x = x ^ 3
>>=	x >>= 3	x = x >> 3
<<=	x <<= 3	x = x << 3

Assignment operators

```
class OpTest2 {  
    public static void main(String[] args) {  
        int a = 4;    int var;    var = a;  
        System.out.println("var using =: " + var);  
        var += a;  
        System.out.println("var using +=: " + var);  
        var *= a;  
        System.out.println("var using *=: " + var);  
        var -= a;  
        System.out.println("var using -=: " + var);  
        var /= a;  
        System.out.println("var using /=: " + var);  
        var %= a;  
        System.out.println("var using %=: " + var);  
    }  
}
```

Comparison operators

Operator	Name	Example
==	Equal to	<code>x == y</code>
!=	Not equal	<code>x != y</code>
>	Greater than	<code>x > y</code>
<	Less than	<code>x < y</code>
>=	Greater than or equal to	<code>x >= y</code>
<=	Less than or equal to	<code>x <= y</code>

Comparison operators

```
class OpTest3 {  
    public static void main(String[] args) {  
        int a = 7, b = 11;  
        System.out.println("a is " + a + " and b is " + b);  
        System.out.println(a == b); // false  
        System.out.println(a != b); // true  
        System.out.println(a > b); // false  
        System.out.println(a < b); // true  
        System.out.println(a >= b); // false  
        System.out.println(a <= b); // true  
    }  
}
```

Logical Operators

Operator	Name	Description	Example
&&	Logical and	Returns true if both statements are true	<code>x < 5 && x < 10</code>
	Logical or	Returns true if one of the statements is true	<code>x < 5 x < 4</code>
!	Logical not	Reverse the result, returns false if the result is true	<code>!(x < 5 && x < 10)</code>

Logical Operators

```
class OpTest4 {  
    public static void main(String[] args) {  
        System.out.println((5 > 3) && (8 > 5)); // true  
        System.out.println((5 > 3) && (8 < 5)); // false  
        System.out.println((5 < 3) || (8 > 5)); // true  
        System.out.println((5 > 3) || (8 < 5)); // true  
        System.out.println((5 < 3) || (8 < 5)); // false  
        System.out.println(!(5 == 3)); // true  
        System.out.println(!(5 > 3)); // false  
    }  
}
```


Shift Operator

Name of operator	Sign	Description
Signed Left Shift	<<	The left shift operator moves all bits by a given number of bits to the left.
Signed Right Shift	>>	The right shift operator moves all bits by a given number of bits to the right.
Unsigned Right Shift	>>>	It is the same as the signed right shift, but the vacant leftmost position is filled with 0 instead of the sign bit.

Shift Operator

- It shifts the underlying bits of an integer to left by the given distance filling the right most bits with zero always.

$X = a \ll b$ means the same as $X = a * 2^b$

a is given Number and b is the shift amount.



Here is an example of 8 bit representation of number 5, and when we left shift it's bit by 3 then the right most 3 bits are filled by zero.

And the number becomes

$$5 * 2^3 = 40.$$

The same thing happens for negative numbers which are represented in 2's complementary notation, for example -5 becomes -40 as follow

Shift Operator

```
public class OperatorExample{  
    public static void main(String args[]){  
        System.out.println(10<<2); //10*2^2=10*4=40  
        System.out.println(10<<3); //10*2^3=10*8=80  
        System.out.println(20<<2); //20*2^2=20*4=80  
        System.out.println(15<<4); //15*2^4=15*16=240  
    }  
}
```

Ternary Operator

```
public class OperatorExample{  
    public static void main(String args[]){  
        int a=2;  
        int b=5;  
        int min=(a<b)?a:b;  
        System.out.println(min);  
    }  
}
```

Bitwise Operator

Operators	Symbol	Uses
Bitwise AND	&	op1 & op2
Bitwise exclusive OR	^	op1 ^ op2
Bitwise inclusive OR		op1 op2
Bitwise Compliment	~	~ op
Bitwise left shift	<<	op1 << op2
Bitwise right shift	>>	op1 >> op2
Unsigned Right Shift Operator	>>> op >>>	number of places to shift

Bitwise Operator

Bitwise AND (&)

x	y	x & y
0	0	0
0	1	0
1	0	0
1	1	1

```
public class BitwiseAndExample
{
    public static void main(String[] args)
    {
        int x = 9, y = 8;
        // bitwise and
        // 1001 & 1000 = 1000 = 8
        System.out.println("x & y = " + (x & y));
    }
}
```

Bitwise Operator

Bitwise exclusive OR (^)

x	y	x ^ y
0	0	0
0	1	1
1	0	1
1	1	0

```
public class BitwiseXorExample
{
    public static void main(String[] args)
    {
        int x = 9, y = 8;
        // bitwise XOR
        // 1001 ^ 1000 = 0001 = 1
        System.out.println("x ^ y = " + (x ^ y));
    }
}
```

Bitwise Operator

Bitwise inclusive OR (|)

x	y	x y
0	0	0
0	1	1
1	0	1
1	1	1

```
public class BitwiseInclusiveOrExample
{
    public static void main(String[] args)
    {
        int x = 9, y = 8;
        // bitwise inclusive OR
        // 1001 | 1000 = 1001 = 9
        System.out.println("x | y = " + (x | y));
    }
}
```


Bitwise Operator

Bitwise Complement (~)
~ (pronounced as the tilde).

X	~X
0	1
1	0

```
public class BitwiseComplimentExample
{
    public static void main(String[] args)
    {
        int x = 2;
        // bitwise compliment
        // ~0010= 1101 = -3
        System.out.println("~x = " + (~x));
    }
}
```

Operator Precedence.

Operator's Precedence in Java

Operators	Precedence
!, +, - (unary Operators)	First (Highest)
*, /, %	Second
+, -	Third
<, <=, >=, >	Fourth
==, !=	Fifth
&&	Sixth
	Seventh
= (assignment Operator)	Lowest

Program Control Structures

Java provides three types of control flow statements.

1. Decision Making statements
 - if statements
 - switch statement
2. Loop statements
 - do while loop
 - while loop
 - for loop
 - for-each loop
3. Jump statements
 - break statement
 - continue statement

If Statement

Java provides three types of control flow statements.

1. Simple if statement `if(condition) {`
2. if-else statement
3. if-else-if ladder `statement 1; //executes when condition is true`
4. Nested if-statement

```
public class Student {  
    public static void main(String[] args) {  
        int x = 10;  
        int y = 12;  
        if(x+y > 20) {
```

```
            System.out.println("x + y is greater than 20");  
        }  
    }  
}
```

if-else statement

Syntax

```
if(condition) {  
    statement 1; //executes when condition is true  
}  
else{  
    statement 2; //executes when condition is false  
}
```

```
public class Student {  
    public static void main(String[] args) {  
        int x = 10; int y = 12;  
        if(x+y < 10) {  
            System.out.println("x + y is less than 10");  
        } else {  
            System.out.println("x + y is greater than 20");  
        }  
    }  
}
```

if-else-if ladder

Syntax

```
if(condition 1) {  
    statement 1; //executes when condition 1 is true  
}  
else if(condition 2) {  
    statement 2; //executes when condition 2 is true  
}  
else {  
    statement 2; //executes when all the conditions are false  
}
```

Nested if-statement ladder

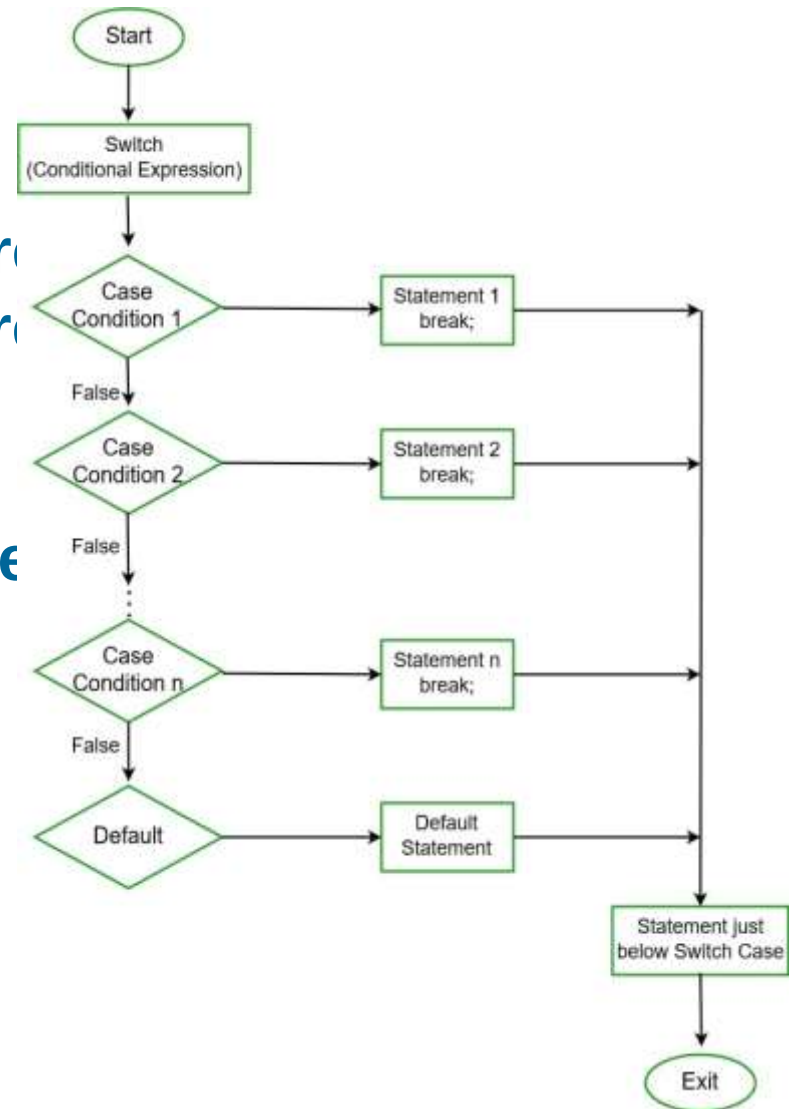
Syntax

```
if(condition 1) {  
    statement 1; //executes when condition 1 is true  
    if(condition 2) {  
        statement 2; //executes when condition 2 is true  
    }  
    else{  
        statement 2; //executes when condition 2 is false  
    }  
}
```

switch statement

Syntax

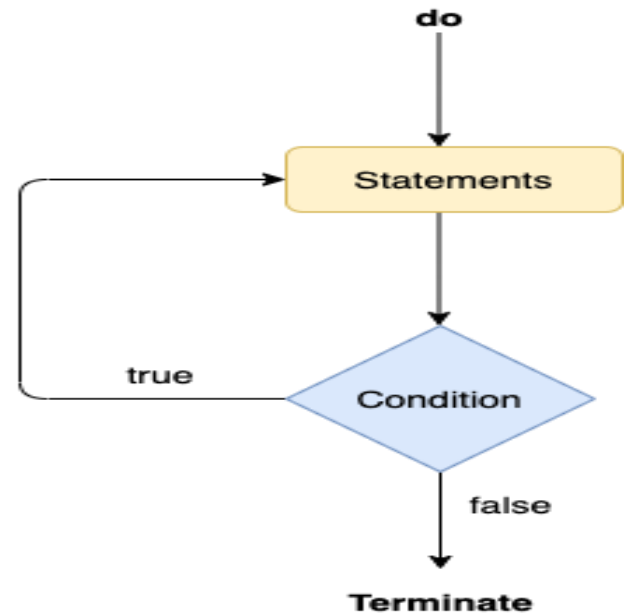
```
switch (expression){  
  case value1:    statement1; break;  
  case value2:    statement2; break;  
  .  
  .  
  case valueN:    statementN; break;  
  default statement;  
}
```



Loop statements: do while loop

Syntax

```
do
{ //statements
} while (condition);
```

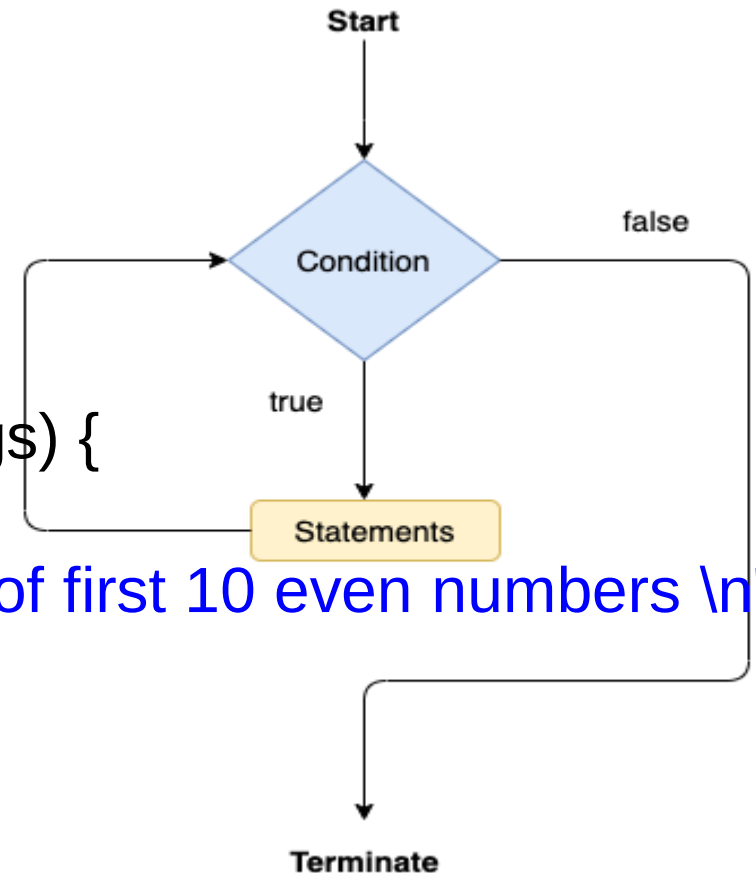


```
public class Calculation {
public static void main(String[] args) {
int i = 0;
System.out.println("Printing the list of first 10 even numbers \
n"); do {
    System.out.println(i);
    i = i + 2;
}while(i<=10);
}
}
```

Loop statements: while loop

Syntax **while**(condition){
 //looping statements
 }

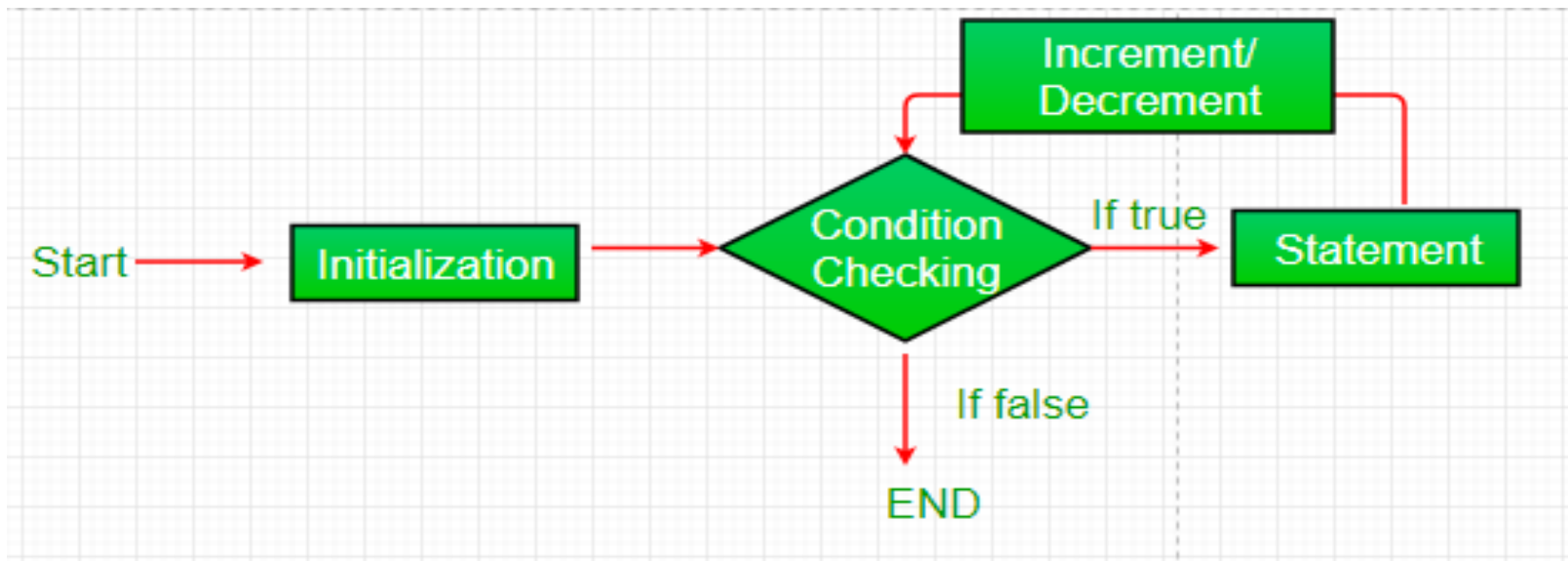
```
public class Calculation {  
    public static void main(String[] args) {  
        int i = 0;  
        System.out.println("Printing the list of first 10 even numbers \n"  
);    while(i<=10) {  
        System.out.println(i);  
        i = i + 2;  
    }  
    }  
}
```



Loop statements: for loop

Syntax

```
for(initialization; condition; increment/decrement)  
{ //block of statements  
}
```



Loop statements: for loop

```
public class Calculation {  
  public static void main(String[] args) {  
    int sum = 0;  
    for(int j = 1; j<=10; j++) {  
      sum = sum + j;  
    }  
  }  
}
```

```
System.out.println("The sum of first 10 natural numbers is " + su  
m);  
}  
}
```

Loop statements: for-each loop

Java provides an enhanced for loop to traverse the data structures like array or collection. In the for-each loop, we don't need to update the loop variable.

Syntax

```
for(data_type var : array_name/collection_name){  
    //statements  
}
```

```
public class Calculation {  
    public static void main(String[] args) {  
        String[] names = {"Java", "C", "C++", "Python", "JavaScript"};  
  
        System.out.println("Printing the content of the array names:\n");  
  
        for(String name:names) {  
            System.out.println(name);  
        }  
    }  
}
```

Jump Statements

Syntax

break;

```
public class BreakExample {  
    public static void main(String[] args) {  
        for(int i = 0; i<= 10; i++) {  
            System.out.println(i);  
            if(i==6) {  
                break;  
            }  
        }  
    }  
}
```

Jump Statements

Syntax **continue;**

```
public class ContinueExample {  
    public static void main(String[] args) {  
        for(int i = 0; i<= 2; i++) {  
            for (int j = i; j<=5; j++) {  
                if(j == 4) {  
                    continue;  
                }  
                System.out.println(j);  
            }  
        }  
    }  
}
```